

PDF expandido

Módulo 3 - Branches, Colaboração, Integração e Infraestrutura Git

Material convertido do curso interativo para leitura linear offline.

Inclui páginas, descrições de imagens, atividades, interações abertas, quiz final com gabarito e glossário.

Módulo 3 — Branches, Colaboração, Integração e Infraestrutura Git

Trilha: Git e GitHub

Módulo 3 - Branches, Colaboração, Integração e Infraestrutura Git



Imagem gerada por IA

Fluxos avançados do Git, destacando branches, colaboração em equipe, integração de alterações por merge ou rebase e a infraestrutura entre repositório local, remoto e GitHub.

Menu lateral

Acesse páginas e acompanhe o ✓ de visita.

Navegação

Use Anterior, Próximo ou os pontos inferiores.

Acordeões

Clique para abrir explicações e práticas.

Simuladores

Teste branches, merge, conflitos e PRs.

Progresso

O SCORM registra páginas visitadas e nota.

Quiz

Finalize com 75% ou mais.

Versão prática reforçada

Além da leitura, cada bloco propõe uma ação verificável para o estudante executar no próprio repositório.

Mapa de práticas: branch -> HEAD -> merge -> conflito -> rebase -> pull request -> fluxo -> infraestrutura -> entrega final.

Autoria

Autoria

Prof. Dr. Eduardo Amadeu Moresi

Prof. Ms. Weldes Lima

Equipe Técnica

Eduardo Amadeu Dutra Moresi

Vilson Hartmann

Mário Braga

Everson Menezes

Isolda Gusmão

Thiago Ribalovsky

Weldes Lima Oliveira

Projeto

Residência em TIC 04B - TIC EM TRILHAS

Direitos Autorais

© 2026 – TIC em Trilhas - Universidade Católica de Brasília UCB

Todos os direitos reservados.

É proibida a reprodução total ou parcial deste material, por qualquer meio, sem autorização expressa da executora.

Sobre o TIC em trilhas

Iniciativa em parceria com TIC em Trilhas e parte integrante do projeto Residência em TIC 04B. Projetos apoiados pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248/91, sendo uma iniciativa do Ministério da Ciência, Tecnologia e Inovações e Softex no âmbito do Programa MCTI Futuro.



EXECUÇÃO



COORDENAÇÃO



INICIATIVA



Régua institucional com os logos do TIC em Trilhas, Universidade Católica de Brasília, Softex, MCTI Futuro, Ministério da Ciência, Tecnologia e Inovação e Governo Federal.

Atividade de fixação

Registre em uma frase: qual decisão técnica desta página você aplicaria no seu repositório?

Escreva sua resposta aqui. Ela fica apenas nesta sessão.

Objetivos de aprendizagem

Objetivos de aprendizagem

Ao concluir este módulo, você será capaz de:

Criar e gerenciar branches em repositórios Git, compreendendo o papel do HEAD e a organização de diferentes linhas de desenvolvimento.

Integrar alterações entre branches, aplicando operações de merge e rebase de acordo com o fluxo de trabalho adotado.

Resolver conflitos de versionamento, analisando divergências no código e registrando decisões de correção de forma clara e documentada.

Participar de fluxos colaborativos no GitHub, utilizando pull requests, commits padronizados, revisão de alterações e publicação do projeto final em repositório remoto.

Missão prática do módulo

Marque as ações que você deverá demonstrar no seu repositório ao final do módulo.

☐ Criar uma branch com nome descritivo.

☐ Integrar alterações por merge e observar o grafo.

☐ Resolver um conflito de merge documentando a decisão.

☐ Abrir ou simular um pull request com checklist.

☐ Escolher um fluxo de trabalho e justificar.

Contrato de prática

Antes de avançar, selecione o que você vai entregar como evidência.

☐ Branch criada com padrão docs/, feature/ ou fix/

☐ Grafo registrado com `git log --oneline --graph --decorate --all`

☐ Pull request com objetivo, alterações e checklist

☐ Conflito resolvido e explicado em Markdown

Introdução

Introdução

Esse módulo tem caráter prático, ao longo do percurso, você será convidado a retomar conceitos, realizar leituras graduais, executar laboratórios práticos, responder a questões de autoavaliação e produzir evidências verificáveis em um repositório. A proposta central é que o conteúdo não seja apenas estudado, mas também experimentado em situações próximas da prática real com Git e GitHub.

Para isso, o módulo utiliza atividades de colaboração simulada, nas quais você poderá vivenciar situações típicas de trabalho em equipe, como divergência de histórico, conflito de merge, revisão de pull request, reorganização de commits e publicação controlada de alterações. A aprendizagem ocorre pela combinação entre explicação conceitual, execução orientada e reflexão sobre decisões técnicas.

O percurso começa com a retomada de conceitos essenciais, como referência, ponteiro, branch e HEAD. Em seguida, você pratica a criação e a alternância de ramificações, integra alterações por merge, resolve conflitos e compara essa estratégia com o rebase. Depois, aproxima-se da lógica de pull requests e revisão por pares, até chegar à possibilidade de consolidar o que foi aprendido em um projeto final.

Esse projeto final é opcional, mas sua realização é altamente recomendada, pois permite colocar em prática os conhecimentos construídos ao longo do módulo. A proposta consiste na criação de um guia de aprendizado interativo, escrito em Markdown e publicado no GitHub. O projeto deve ser organizado por branches, submetido a pelo menos um pull request, revisado por pares ou por autoavaliação estruturada, além de conter o registro de um conflito resolvido e uma reflexão final sobre o fluxo de trabalho adotado.

Ao desenvolver esse projeto, você transforma o GitHub em objeto e ambiente de aprendizagem ao mesmo tempo: aprende usando a própria plataforma, documenta o processo, registra escolhas técnicas e produz um artefato que pode ser reutilizado em seu portfólio acadêmico ou profissional. Embora não seja obrigatório, o projeto representa uma oportunidade importante de consolidar a aprendizagem, demonstrar o processo de desenvolvimento e tornar visíveis as decisões tomadas durante a prática.

Rota de prática do módulo

1. Branch

2. Merge

3. Conflito

4. PR

5. Projeto

Projeto opcional recomendado: guia de aprendizado em Markdown publicado no GitHub, com branch, pull request, conflito resolvido e reflexão final.

Projeto opcional guiado

Construa um guia de aprendizado em Markdown. Cada etapa abaixo vira um commit ou pull request.

1. Criar repositório guia-git-colaborativo
2. Criar branch docs/estrutura-guia
3. Adicionar seções sobre branch, merge e PR
4. Criar um conflito controlado no README
5. Abrir PR, revisar e integrar

1. Arquitetura das ramificações no Git

1. Arquitetura das ramificações no Git

A ramificação é uma das características mais expressivas do Git porque torna viável a coexistência de múltiplas linhas de desenvolvimento dentro do mesmo repositório. Estudantes podem imaginar uma branch como uma pasta duplicada ou uma cópia independente do projeto. Essa metáfora é compreensível, mas tecnicamente inadequada. Uma branch é, em essência, uma referência nomeada para um commit. O custo de criar uma branch é muito baixo porque o Git não duplica todos os arquivos do projeto; ele cria ou atualiza um ponteiro que indica qual commit representa a ponta daquela linha de desenvolvimento.

A branch principal, tradicionalmente chamada master e atualmente denominada main em muitos ambientes, representa a linha de desenvolvimento que a equipe decide tratar como referência central. A escolha do nome main não altera a mecânica interna do Git, mas tem relevância cultural e operacional. O importante é que a main seja compreendida como convenção de governança: ela deve expressar um estado confiável, revisado e, quando possível, pronto para execução ou publicação. Em projetos acadêmicos, manter a main estável ensina disciplina de integração e evita que experimentos incompletos sejam confundidos com versões entregáveis.

A ponta de uma branch, frequentemente referida como tip, é o commit mais recente apontado por aquela referência. Quando você realiza um novo commit estando em uma branch ativa, o Git cria um objeto de commit cujo pai é o commit anterior e move a referência da branch para apontar para esse novo commit. O histórico não "anda" sozinho: o deslocamento da branch ocorre porque o Git atualiza a referência. Essa noção é central para compreender por que alternar de branch modifica o conteúdo visível no diretório de trabalho.

O ponteiro HEAD desempenha papel fundamental na determinação do contexto corrente. Em estado normal, HEAD aponta simbolicamente para uma branch, e essa branch aponta para um commit. Quando você está "na branch feature-menu", tecnicamente significa que HEAD referencia feature-menu, e feature-menu referencia um determinado commit. Ao criar um commit, o Git consulta HEAD para descobrir qual referência deve ser movida. Essa cadeia de referências — HEAD, branch, commit — deve ser internalizada, pois permite interpretar mensagens do Git e diagnosticar situações como detached HEAD, divergência local-remota e conflitos de integração.

A relação entre branch, commit e HEAD pode ser representada por um modelo mental de três níveis. No primeiro nível está você, que executa comandos e altera arquivos. No segundo nível estão as referências, que dão nomes humanos a posições no grafo de commits. No terceiro nível está o grafo propriamente dito, formado por commits e relações de ancestralidade. O erro comum é confundir o nome da branch com os arquivos do projeto. A branch não contém os arquivos; ela aponta para um commit que aponta para uma árvore que descreve os arquivos.

A arquitetura de ramificações permite isolamento. Uma branch de funcionalidade pode conter experimentos, protótipos, textos incompletos, código em teste ou documentação ainda não revisada, sem alterar a linha principal. Isso é particularmente importante no ensino de graduação porque reduz o medo

de experimentar. Você aprende que erros podem ser isolados, comparados, descartados ou incorporados posteriormente.

Entretanto, a facilidade de criar branches não elimina a necessidade de método. Muitos projetos iniciantes acumulam branches sem nome claro, histórico confuso e commits grandes demais. A prática profissional recomenda nomes descritivos, escopo limitado e integração frequente. Uma branch longa e pouco sincronizada tende a gerar conflitos maiores; uma branch curta, com objetivo bem delimitado, tende a ser revisada com mais facilidade.

No contexto do GitHub, branches ganham uma dimensão comunicacional, pois são associadas a pull requests, checks, revisões e discussões. A branch local é o seu espaço de trabalho; a branch remota publicada é a base para que outros acompanhem suas alterações; o pull request é o ambiente de revisão que transforma uma linha de trabalho privada em proposta pública de mudança.

Para consolidar essa compreensão, manipule branches no terminal, visualize o grafo de commits e compare o estado do diretório de trabalho ao alternar entre referências. A visualização do histórico com `git log --oneline --graph --decorate` é recomendada como rotina de estudo.

```
git branch feature/introducao # cria uma referência, mas não troca para ela
git switch feature/introducao # altera HEAD para apontar simbolicamente para a branch
git log --oneline --graph --decorate --all # visualiza commits, branches e relações de ancestralidade
```

Simulador: branch, commit e HEAD

Clique nos comandos e observe o que muda no modelo mental.



main

HEAD → main

`git branch feature/intro`

`git switch feature/intro`

`git commit`

`git merge feature/intro`

Estado inicial: HEAD aponta para main.

Laboratório orientado: criando sua primeira branch

Copie e execute os comandos. Depois marque cada verificação.

```
git status
git switch -c docs/introducao
echo "## Introdução" >> guia.md
git add guia.md
git commit -m "docs: adiciona introducao do guia"
git log --oneline --graph --decorate --all
```

- ☐ Verifiquei em qual branch estou.
- ☐ Criei um commit na branch nova.
- ☐ Consegui visualizar HEAD e a branch no grafo.

2. Gestão de branches: comandos, convenções e práticas

2. Gestão de branches: comandos, convenções e práticas

A gestão de branches envolve operações de criação, alternância, listagem, renomeação, exclusão e sincronização com referências remotas. O comando `git branch`, quando executado sem argumentos, lista as branches locais e indica com asterisco aquela que está ativa. Quando executado com um nome, cria uma nova referência para o commit atual, mas não altera o contexto de trabalho. Essa diferença é importante: criar branch e trocar para branch são operações distintas. O comando `git switch` foi introduzido para tornar mais explícita a alternância de contexto, enquanto `git checkout` permanece amplamente utilizado e acumula funções históricas.

Para você, iniciante, priorize `git switch` em atividades didáticas. A sintaxe `git switch -c feature/nome` cria e troca para a nova branch, tornando o fluxo mais legível. O uso de `git checkout -b` também deve ser conhecido, pois aparece em muitos tutoriais, projetos legados e respostas de fóruns técnicos. Contudo, `checkout` é um comando multifuncional e, por isso, mais propenso a confusão. Em uma trilha autoinstrucional, a clareza semântica dos comandos reduz a carga cognitiva e facilita a interpretação de erros.

A criação de branches deve seguir convenções de nomenclatura. Nomes como "teste", "nova", "minha-branch" ou "final" não comunicam finalidade. Prefira padrões como `feature/descricao`, `fix/descricao`, `docs/descricao`, `refactor/descricao` ou `experiment/descricao`. O prefixo ajuda a identificar a natureza da alteração, enquanto a descrição curta indica o objetivo. Por exemplo, `feature/guia-markdown` comunica melhor que `alteracoes-eduardo`.

A listagem de branches pode ser enriquecida por opções como `git branch -v`, que mostra o último commit associado a cada branch, e `git branch --merged`, que indica branches já integradas à branch atual. Após uma branch ser incorporada por `merge` ou `pull request`, ela pode ser removida localmente com `git branch -d nome`. A opção `-d` é segura porque impede remoção de branches ainda não mescladas. A opção `-D` força a exclusão, devendo ser usada apenas quando você compreende que está descartando uma linha de desenvolvimento não integrada.

A distinção entre branch local e branch remota é uma fonte recorrente de dificuldades. Ao clonar um repositório, o Git cria branches locais e referências de rastreamento remoto, como `origin/main`. A referência `origin/main` não é uma branch local editável diretamente; ela representa a última posição conhecida da branch `main` no remoto `origin`. Quando você executa `git fetch`, essas referências de rastreamento são atualizadas.

Em fluxos com GitHub, a publicação de uma branch local costuma envolver `git push -u origin nome-da-branch`. A opção `-u` configura o upstream, isto é, a relação de rastreamento entre a branch local e a branch remota. Depois disso, comandos como `git push` e `git pull` podem operar sem especificar explicitamente `origin` e o nome da branch.

A gestão adequada de branches inclui limpeza periódica. Em repositórios de longa duração, branches antigas podem confundir estudantes e revisores, especialmente quando permanecem no GitHub após

pull requests encerrados. A exclusão de uma branch remota pode ser feita com `git push origin --delete nome-da-branch`, desde que você esteja seguro de que a contribuição foi integrada ou descartada.

A gestão de branches também tem relação com granularidade de trabalho. Uma branch deve representar uma unidade compreensível de mudança: uma funcionalidade, uma correção, uma seção de documentação, uma atividade de laboratório ou uma pequena refatoração. Branches que acumulam muitas finalidades tornam a revisão difícil e aumentam a probabilidade de conflitos.

```
git switch -c docs/estrutura-guia git branch -v git push -u origin docs/estrutura-guia git  
branch --merged git branch -d docs/estrutura-guia git push origin --delete docs/  
estrutura-guia
```

Comando	Finalidade pedagógica	Risco ou observação
<code>git branch nome</code>	Criar uma referência local no commit atual.	Não troca automaticamente para a nova branch.
<code>git switch nome</code>	Alternar o contexto de trabalho para uma branch existente.	Pode falhar se houver alterações locais conflitantes.
<code>git switch -c nome</code>	Criar e alternar para uma branch em um único passo.	Recomendado para iniciantes pela clareza semântica.
<code>git branch -d nome</code>	Excluir branch já mesclada.	Operação segura; impede descarte acidental de trabalho não integrado.
<code>git branch -D nome</code>	Excluir branch à força.	Deve ser usada apenas com compreensão do descarte.
<code>git push -u origin nome</code>	Publicar branch e configurar upstream.	Estabelece relação entre branch local e remota.

Prática: nomeie a branch corretamente

Escolha o melhor nome para uma branch cujo objetivo é criar a seção de referências do guia.

final

minha-branch

docs/referencias

teste2

Associação rápida: comando e finalidade

git switch -c nome

git branch -v

git branch -d nome

git push -u origin nome

Gerador de comando de branch

Modelo de comando para criar uma branch com tipo e descrição coerentes:

```
git switch -c docs/descricao
```

Exemplos: docs/estrutura-guia, feature/simulador-merge, fix/correcao-readme.

3. HEAD, detached HEAD e recuperação de referências

3. HEAD, detached HEAD e recuperação de referências

O HEAD é uma referência especial que informa ao Git qual é o ponto de partida para novas operações. Em estado normal, HEAD aponta simbolicamente para uma branch. Quando você executa `git status`, a mensagem "On branch main" indica essa relação. Ao realizar um commit, o Git cria o novo objeto e move a branch apontada por HEAD para o novo commit. Compreender HEAD ajuda a interpretar por que o mesmo comando pode produzir efeitos diferentes dependendo do contexto em que é executado.

O estado detached HEAD ocorre quando HEAD deixa de apontar para uma branch e passa a apontar diretamente para um commit específico. Isso pode acontecer quando você executa `git checkout <hash>`, `git switch --detach <hash>`, acessa uma tag ou navega para uma referência remota sem criar branch local. Esse estado não é necessariamente erro. Ele é útil para inspeção de versões antigas, testes temporários e comparação de comportamentos. O problema surge quando você realiza commits nesse estado e depois troca para outra branch sem criar uma referência para preservar o trabalho.

A explicação do detached HEAD deve evitar alarmismo, mas enfatizar responsabilidade. A mensagem do Git costuma informar que você está em estado destacado e sugerir criação de branch. Em uma atividade didática, acesse um commit antigo com checkout, observe os arquivos, execute `git status` e retorne à main. Em seguida, crie uma branch a partir daquele ponto para compreender a diferença entre inspeção e continuidade de trabalho.

A recuperação de trabalho em detached HEAD pode ser feita criando uma branch antes de sair do estado ou usando o reflog para localizar commits perdidos. O comando `git switch -c recuperacao` cria uma branch apontando para o commit atual, preservando os commits feitos naquele contexto. Se você já saiu do estado, `git reflog` lista movimentos recentes de HEAD, permitindo identificar o hash do commit e criar uma branch a partir dele.

O reflog, embora poderoso, deve ser apresentado como mecanismo local e temporário, não como substituto de boas práticas. Ele registra atualizações de referências no repositório local e pode ser limpo por políticas de expiração ou coleta de lixo. Em repositórios compartilhados, outros colaboradores não acessam seu reflog local. Portanto, a prevenção continua sendo melhor que a recuperação.

HEAD também aparece em expressões relativas, como `HEAD~1`, `HEAD~2` e `HEAD^`, que indicam ancestrais do commit atual. Essas expressões são usadas em comandos como `git reset`, `git rebase -i HEAD~4` e `git show HEAD~1`. Antes de executar comandos que modificam histórico, pergunte-se: "A que commit HEAD aponta agora?" Essa pergunta simples reduz riscos em rebase, reset e merge.

A relação entre HEAD e branches remotas também precisa ser esclarecida. Ao executar `git checkout origin/main`, você geralmente entra em detached HEAD, pois `origin/main` é referência de rastreamento, não branch local de trabalho. Para trabalhar a partir dela, crie uma branch local baseada na referência remota ou atualize a branch local existente.


```
git log --oneline --decorate --graph --all git switch --detach HEAD~1 git status # se fizer
alterações e quiser preservar: git switch -c experimento-a-partir-de-versao-antiga # se
já perdeu o ponto: git reflog git switch -c recuperacao <hash-encontrado>
```

Simulador: detached HEAD

Selecione uma ação e veja se ela é segura.

`git switch --detach HEAD~1`

`git switch -c recuperacao`

`git reflog`

Aguardando escolha.

Diagnóstico rápido: estou em detached HEAD?

Alternativas:

- HEAD detached at a1b2c3d - exige cuidado.
- On branch docs/guia - correto: você está em uma branch local de trabalho.
- HEAD detached from origin/main - exige cuidado.

4. Integração de código com merge

4. Integração de código com merge

A integração de código é o processo pelo qual alterações produzidas em linhas distintas de desenvolvimento são incorporadas a uma linha comum. No Git, a forma clássica de integração é o merge. O comando `git merge` solicita que o Git incorpore commits de uma branch de origem à branch atualmente ativa. A operação depende da relação histórica entre os commits envolvidos. Se a branch de destino não avançou desde que a branch de origem foi criada, o Git pode realizar um fast-forward, apenas movendo a referência da branch de destino para o commit mais recente da origem.

O fast-forward é a integração mais simples. Imagine que `main` aponta para o commit A e `feature` aponta para o commit C, sendo C descendente direto de A. Se `main` não recebeu outros commits, integrar `feature` em `main` não exige criar um commit novo; basta mover `main` para C. O histórico permanece linear. Essa operação é eficiente e clara, mas pode ocultar o fato de que houve uma branch temporária.

O merge de três vias ocorre quando a branch de destino e a branch de origem possuem commits diferentes desde o ancestral comum. Nesse caso, o Git identifica três snapshots: o ancestral comum, a ponta da branch atual e a ponta da branch a ser integrada. A partir dessa comparação, tenta construir um novo snapshot que combine as alterações. Se não houver alterações incompatíveis, cria um commit de merge com dois pais. Visualizado em grafo, ele mostra o ponto em que linhas paralelas se reencontram.

A escolha entre preservar commits de merge ou manter histórico linear depende do fluxo de trabalho adotado. Em contextos educacionais, é útil praticar ambos. O merge com commit explícito facilita visualizar a existência de branches e ajuda a compreender o grafo. A ferramenta `git log --graph` torna essas diferenças visíveis.

O merge deve ser executado com diretório de trabalho limpo. Verifique `git status` antes de integrar. Se houver alterações não commitadas, você deve decidir entre commitá-las, descartá-las ou guardá-las temporariamente com `stash`.

A integração por merge também exige comunicação. Quando uma branch é mesclada, não apenas arquivos são combinados; decisões de projeto são incorporadas. Em um pull request, a descrição do que está sendo mesclado deve explicar objetivo, escopo, arquivos relevantes, testes executados e possíveis impactos.

Há situações em que o merge é preferível ao rebase. Branches públicas, compartilhadas por vários colaboradores ou associadas a pull requests já discutidos devem preservar histórico para evitar divergência entre clones. O merge conserva a cronologia das contribuições e reduz riscos de reescrita. Em equipes que valorizam rastreabilidade completa, commits de merge registram pontos de decisão.

O `git merge` pode ser abortado em determinadas situações. Se um merge gerar conflitos e você perceber que iniciou a operação por engano, `git merge --abort` tenta restaurar o estado anterior.

Ao final desta unidade, você deve ser capaz de executar um fluxo completo: criar branch, produzir commits, retornar à main, integrar a branch, reconhecer se ocorreu fast-forward ou merge commit, visualizar o grafo e explicar o resultado.

```
git switch main git status git merge feature/guia-markdown git log --oneline --graph --
decorate --all # em caso de conflito iniciado por engano: git merge --abort
```

Simulador: fast-forward ou merge commit?

Escolha o cenário e observe a integração.

Cenário A: main não avançou

Cenário B: main e feature avançaram

Selecione um cenário.

Planeje o merge antes de executar

Critério de decisão:

- Se há alterações não commitadas, primeiro resolva suas alterações locais: commit, descarte ou stash. Evite iniciar merge com diretório sujo.
- Se main não avançou, o cenário provável é fast-forward.
- Se main também avançou, o cenário provável é merge de três vias: visualize o grafo e prepare-se para possível commit de merge.

5. Resolução de conflitos de merge

5. Resolução de conflitos de merge

Conflitos de merge ocorrem quando o Git não consegue determinar automaticamente como combinar alterações concorrentes. O caso didático mais simples envolve duas branches que modificam a mesma linha do mesmo arquivo de maneiras diferentes. Ao tentar integrar, o Git interrompe a operação e marca o arquivo com delimitadores de conflito. Esses marcadores indicam a versão da branch atual, a separação entre versões e a versão da branch sendo integrada. O Git identificou a incompatibilidade, mas a escolha semântica depende de você.

Os marcadores tradicionais são <<<<<<, ===== e >>>>>>. A região entre <<<<<< HEAD e ===== representa a versão existente na branch atual. A região entre ===== e >>>>>> nome-da-branch representa a versão da branch integrada. Resolver o conflito consiste em editar o arquivo até que ele represente a versão final desejada, removendo os marcadores e preservando o conteúdo correto. Depois, execute git add para informar que o arquivo foi resolvido e git commit para concluir o merge.

Em termos pedagógicos, a resolução de conflitos é um dos momentos mais valiosos do curso. Ela obriga você a abandonar a memorização de comandos e analisar conteúdo, contexto e consequência. Em vez de escolher automaticamente "minha versão" ou "versão deles", deve perguntar qual alteração atende ao objetivo do projeto.

Conflitos podem ocorrer em arquivos de texto, documentação, código, configurações e até arquivos gerados. Arquivos binários apresentam desafios maiores porque o Git não consegue mesclar linha a linha. Isso reforça a importância de boas práticas já estudadas, como uso de .gitignore para evitar versionamento de artefatos gerados, logs, dependências e arquivos temporários.

Ferramentas visuais de merge podem apoiá-lo, especialmente quando há vários arquivos conflitantes. Editores como Visual Studio Code apresentam botões para aceitar a versão atual, aceitar a versão recebida ou combinar alterações. Entretanto, o uso dessas ferramentas deve ser acompanhado de compreensão dos marcadores. Recomenda-se primeiro resolver um conflito manualmente em editor de texto e só depois experimentar ferramentas visuais.

O comando git status é essencial durante conflitos. Ele indica quais arquivos estão em conflito e quais já foram marcados como resolvidos. Após editar um arquivo, git diff pode mostrar a diferença restante; após git add, o arquivo deixa a categoria de conflito não resolvido. A sequência — status, edição, diff, add, status, commit — deve ser praticada até se tornar rotina.

A prevenção também é importante. Conflitos pequenos e frequentes são mais fáceis de resolver que conflitos grandes e tardios. Branches longas, alterações extensas e falta de sincronização com a main aumentam a complexidade. Integração frequente e comunicação prévia sobre arquivos críticos reduzem riscos.

A documentação da resolução é parte do aprendizado. No projeto final, inclua um arquivo conflito-resolvido.md explicando qual conflito foi criado, por que ele ocorreu, quais alternativas existiam e qual decisão foi tomada.

```
<<<<<<< HEAD Texto da versão atual na branch main. ===== Texto da versão
desenvolvida na branch feature/contato. >>>>>>> feature/contato git status # editar o
arquivo e remover os marcadores git diff git add README.md git status git commit
```

Prática: resolva o conflito

Escolha a versão final mais adequada para um README em que duas branches alteraram a mesma frase.

```
<<<<<<< HEAD
Este guia explica branches.
=====
Este guia explica branches e pull requests.
>>>>>>> feature/pr
```

Manter apenas "Este guia explica branches" Combinar: "Este guia explica branches e pull requests."

Deixar os marcadores para mostrar o conflito

Removi os marcadores de conflito.

Rodei git status.

Executei git add no arquivo resolvido.

Fiz o commit do merge.

Editor de resolução de conflito

Reescreva a versão final sem os marcadores <<<<<<, ===== e >>>>>>.

Este guia explica branches e pull requests.

Validação: boa resolução não mantém marcadores de conflito e preserva a versão final escolhida.

6. Rebase, histórico linear e ética da reescrita

6. Rebase, histórico linear e ética da reescrita

O rebase é uma estratégia de integração que reaplica commits de uma branch sobre outro ponto de partida. Enquanto o merge preserva explicitamente a convergência de linhas de desenvolvimento, o rebase reorganiza a aparência do histórico para que os commits pareçam ter sido criados a partir de uma base mais recente. Essa reorganização produz um histórico linear, frequentemente considerado mais legível. Contudo, a legibilidade tem custo: os commits reaplicados recebem novos hashes porque seus pais mudam. Portanto, rebase não é apenas deslocamento visual; é reescrita de histórico.

A operação típica ocorre quando uma branch de funcionalidade foi criada a partir de main, mas main avançou com novos commits. Ao executar `git rebase main` estando na branch de funcionalidade, o Git identifica os commits exclusivos da branch, remove-os temporariamente, atualiza a base para a main atual e reaplica cada commit. O resultado é uma sequência linear em que a funcionalidade aparece como se tivesse começado a partir da versão mais recente da main.

A comparação entre merge e rebase deve ser feita em termos de finalidade, não de superioridade absoluta. Merge preserva contexto histórico e é seguro para branches compartilhadas. Rebase melhora linearidade e é apropriado para branches locais ou privadas, antes de publicação ou quando a equipe explicitamente concorda com essa prática. Dizer que rebase é "melhor" porque deixa o log bonito é simplificação perigosa.

⚠ Regra de ouro: não faça rebase de branches públicas compartilhadas sem acordo explícito. Outros clones podem depender dos hashes antigos e a alteração unilateral do histórico impacta colaboradores.

O rebase interativo, executado com `git rebase -i`, permite editar a sequência de commits antes da publicação. Você pode combinar commits pequenos com squash, alterar mensagens com `reword`, remover commits acidentais com `drop` ou reorganizar commits. Por exemplo, quatro commits chamados "teste", "ajuste", "corrige de novo" e "agora vai" podem ser combinados em um commit claro, como "docs: estrutura guia de aprendizado".

O squash deve ser usado com discernimento. Commits atômicos e informativos não precisam ser sempre combinados. Se cada commit representa uma decisão lógica distinta, preservá-los pode ser melhor. Se vários commits representam microajustes sem valor histórico independente, squash pode melhorar legibilidade.

Durante rebase, conflitos podem surgir. Diferentemente do merge, o Git reaplica commits um a um; se um commit entra em conflito, o processo pausa. Resolva o conflito, execute `git add` e continue com `git rebase --continue`. Se decidir abandonar a operação, pode usar `git rebase --abort`.

O comando `git log --oneline --graph --decorate --all` ajuda a visualizar antes e depois do rebase. Uma atividade eficaz consiste em anotar os hashes de commits antes do rebase interativo, executar `squash` e comparar o resultado. Essa experiência torna concreta a afirmação de que o rebase reescreve histórico.

```
git switch feature/guia git fetch origin git rebase origin/main # se houver conflito:
editar arquivos, depois git add arquivo.md git rebase --continue # para desistir git
rebase --abort # rebase interativo para organizar últimos quatro commits git rebase -i
HEAD~4
```

Aspecto	Merge	Rebase
Histórico	Preserva bifurcações e pontos de integração.	Reescreve commits para criar linha mais linear.
Hashes	Mantém hashes dos commits integrados.	Cria novos hashes para commits reaplicados.
Uso recomendado	Branches públicas, integração revisada e rastreável.	Branches locais ou privadas antes da publicação.
Conflitos	Normalmente resolvidos em um ponto de integração.	Podem aparecer commit a commit durante a reaplicação.
Risco	Histórico pode ficar mais ramificado.	Reescrita indevida pode afetar colaboradores.

Decisor: merge ou rebase?

Leia o contexto e escolha a estratégia mais segura.

Sua branch é local, ainda não foi publicada e contém quatro commits pequenos chamados "teste", "ajuste", "corrige" e "final".

Usar rebase interativo / squash

Usar merge commit

Trocar cenário

Antes de usar rebase, responda

☐ A branch é privada ou há acordo explícito da equipe.

☐ Entendo que os hashes serão recriados.

☐ Tenho diretório de trabalho limpo.

Critério: só siga com cautela quando as três condições estiverem atendidas.

7. Pull requests e revisão de código no GitHub

7. Pull requests e revisão de código no GitHub

O pull request é o mecanismo pelo qual uma branch publicada se transforma em proposta de alteração para outra branch, geralmente main. Apesar do nome, no GitHub o pull request não é apenas um pedido técnico para puxar commits; ele é um espaço de discussão, revisão, validação automática e documentação de decisões. O autor descreve o objetivo da mudança, revisores examinam arquivos, comentários são registrados, ajustes são solicitados, testes automatizados podem ser executados e, ao final, a contribuição é integrada ou rejeitada.

A anatomia de um pull request inclui título, descrição, lista de commits, diferenças entre arquivos, comentários gerais, comentários em linhas específicas, checks, revisores, labels e histórico de eventos. Aprenda a escrever títulos informativos, como "docs: adiciona seção sobre merge", em vez de "trabalho final". A descrição deve explicar o que foi feito, por que foi feito, como testar ou revisar e quais limitações permanecem.

A revisão por pares, quando bem conduzida, tem função dupla: melhorar o artefato e ensinar. Revisores podem identificar erros, inconsistências, riscos de segurança, trechos confusos e oportunidades de simplificação. Você, como autor, aprende a defender decisões, aceitar sugestões e ajustar o trabalho sem personalizar críticas.

Em uma trilha autoinstrucional, a revisão pode ser simulada quando não houver colegas disponíveis. Use uma rubrica de autoavaliação, abra o próprio pull request, preencha checklist e registre comentários. No entanto, sempre que possível, pares devem revisar mutuamente seus projetos.

O pull request também favorece rastreabilidade. Ao longo do tempo, é possível reconstruir por que uma decisão foi tomada, quem revisou, quais alternativas foram consideradas e quais testes foram realizados. Essa rastreabilidade é valiosa para manutenção, auditoria e aprendizagem.

A integração do pull request pode ocorrer por diferentes estratégias: Merge commit mantém o histórico da branch e cria um commit de integração. Squash and merge combina todos os commits da branch em um único commit na branch de destino. Rebase and merge reaplica commits na base de destino sem criar commit de merge. Para aprendizagem de grafo, merge commit é didaticamente útil.

O pull request deve ser pequeno o suficiente para revisão humana. Contribuições grandes, com muitos arquivos e objetivos misturados, dificultam feedback. Limite PRs por seção de documentação ou por funcionalidade pequena.

A revisão também deve seguir etiqueta profissional. Comentários devem focar no artefato, não na pessoa; sugestões devem ser específicas; discordâncias devem ser justificadas. Você, como autor, deve responder de forma objetiva e explicar quando não adotar uma sugestão.

Ao final desta unidade, abra um pull request de sua branch para main, preencha descrição estruturada, solicite ou simule revisão, responda a comentários e integre a contribuição. A entrega deve incluir link do pull request e breve reflexão sobre o que foi aprendido.

Modelo de descrição de pull request:

Objetivo Descrever claramente a mudança proposta. **## Alterações realizadas** - Item 1 - Item 2 - Item 3 **## Como revisar** Informar arquivos principais, comandos executados ou critérios de leitura. **## Riscos ou limitações** Indicar pontos que ainda precisam de atenção. **## Checklist** - [] A branch tem escopo claro. - [] Os commits seguem padrão combinado. - [] O README foi atualizado quando necessário. - [] Não há arquivos sensíveis ou temporários. - [] O histórico foi revisado antes do merge.

Montador de descrição de Pull Request

Montar descrição

Preencha os campos para gerar a descrição.

Branch tem escopo claro.

Commits seguem padrão combinado.

README foi atualizado se necessário.

Não há arquivos sensíveis.

Checklist de revisão por pares

Use este checklist para revisar o PR de um colega ou fazer autoavaliação.

Título comunica o objetivo?

Descrição explica como revisar?

Commits são pequenos e claros?

Não há arquivos sensíveis?

README foi atualizado?

8. Fluxos de trabalho: centralizado, Git Flow, GitHub Flow e GitLab Flow

8. Fluxos de trabalho: centralizado, Git Flow, GitHub Flow e GitLab Flow

Um fluxo de trabalho define como uma equipe organiza branches, integra mudanças, prepara releases e protege a linha principal de desenvolvimento. O Git permite muitos arranjos; essa flexibilidade é poderosa, mas pode confundir equipes iniciantes. Este módulo apresenta quatro modelos: fluxo centralizado, Git Flow, GitHub Flow e GitLab Flow. O objetivo não é impor um modelo universal, mas ensinar critérios de escolha.

O fluxo centralizado é o mais simples. Todos trabalham em torno de uma branch principal, e a colaboração ocorre por commits e sincronizações frequentes. Em equipes muito pequenas ou em exercícios introdutórios, pode funcionar. Contudo, quando vários estudantes editam os mesmos arquivos, a ausência de branches de funcionalidade aumenta conflitos e reduz isolamento.

O Git Flow é um modelo mais estruturado, originalmente concebido para projetos com ciclos de release. Ele distingue branches como main para produção, develop para integração, feature branches para funcionalidades, release branches para preparação de versões e hotfix branches para correções urgentes. Sua vantagem é disciplinar projetos que mantêm versões numeradas. Sua desvantagem é a complexidade: para você, iniciante, muitas categorias de branch podem gerar sobrecarga.

O GitHub Flow é mais simples e se ajusta bem a ambientes de entrega contínua e projetos web. A regra básica: a main deve estar sempre em estado publicável; qualquer mudança é feita em branch curta; abre-se um pull request; há revisão e verificação; depois do merge na main, a mudança pode ser publicada. Esse modelo é pedagogicamente adequado porque conecta branch, pull request, revisão e integração sem exigir múltiplas branches permanentes.

O GitLab Flow procura equilibrar simplicidade e controle por ambientes. Ele pode utilizar branches associadas a ambientes, como production, staging ou pre-production. Em cursos introdutórios, seu valor está em mostrar que fluxos de trabalho podem refletir ambientes reais de entrega.

A escolha do fluxo deve ser justificada por critérios. Para um projeto individual de guia Markdown, uma main protegida e branches de documentação com pull requests são suficientes. Para um projeto de software com releases semestrais, Git Flow pode fazer sentido. Para uma aplicação web com deploy frequente, GitHub Flow é mais direto.

Em repositórios GitHub, políticas de proteção de branch podem reforçar o fluxo escolhido. A proteção pode impedir commits diretos na main, exigir pull request, exigir aprovação, exigir checks verdes e restringir quem pode fazer merge.

Para o projeto final do módulo, adote uma versão simplificada do GitHub Flow: crie uma branch para cada seção do guia, publique a branch, abra pull request, revise, integre à main e remova branch após merge. Documente por que esse fluxo foi escolhido e quais seriam as limitações se o projeto crescesse.

Fluxo	Características	Quando usar	Limitações didáticas
Centralizado	Trabalho direto na branch principal, baixa cerimônia.	Exercícios pequenos e equipes muito reduzidas.	Pouca prática de pull request e isolamento.
Git Flow	Branches main, develop, feature, release e hotfix.	Produtos com versões planejadas e ciclos de release.	Complexidade elevada para iniciantes.
GitHub Flow	Branches curtas, pull request e main sempre publicável.	Projetos web, documentação, entregas frequentes e ensino inicial.	Exige disciplina de revisão e testes.
GitLab Flow	Integra branches e ambientes, como staging e production.	Equipes com múltiplos ambientes de entrega.	Pode ser excessivo para projetos acadêmicos simples.

Escolha o fluxo adequado

Para um guia Markdown individual com revisão por PR, qual fluxo é mais adequado?

Git Flow completo

GitHub Flow simplificado

GitLab Flow com staging/production

Sem branches

Simule a escolha do fluxo

- **Guia Markdown individual com revisão simples:** use GitHub Flow simplificado.
- **Produto com versões planejadas e hotfix:** considere Git Flow.
- **Aplicação web com entregas frequentes:** use GitHub Flow.

9. Infraestrutura Git: remotes, protocolos, autenticação e segurança

9. Infraestrutura Git: remotes, protocolos, autenticação e segurança

A colaboração distribuída requer transferência de objetos entre repositórios. Em Git, um remote é um apelido para outro repositório, frequentemente hospedado em uma plataforma como GitHub. O nome origin é convenção para o remoto principal criado no clone ou configurado manualmente. Embora você muitas vezes associe remote exclusivamente ao GitHub, o conceito é mais amplo: um remoto pode ser outro diretório local, um servidor via SSH, um endpoint HTTPS, um repositório bare em uma máquina institucional ou uma plataforma corporativa.

Os principais protocolos de acesso incluem Local, SSH, HTTP/HTTPS e o protocolo Git. O protocolo Local usa caminhos do sistema de arquivos e é útil para laboratórios, backups locais e demonstrações de repositórios bare. O SSH oferece autenticação por chave criptográfica e canal seguro, sendo comum em servidores e plataformas de hospedagem. O HTTPS é amplamente usado em ambientes corporativos por atravessar firewalls e integrar autenticação por tokens.

No GitHub, a autenticação por HTTPS geralmente utiliza tokens de acesso em vez de senha de conta. Tokens podem ter escopo limitado, expiração e revogação, o que melhora a segurança em comparação com senhas estáticas. O SSH utiliza par de chaves pública e privada. A chave pública é cadastrada no GitHub ou no servidor; a chave privada permanece no computador do usuário e deve ser protegida com passphrase.

Um repositório bare é um repositório sem diretório de trabalho, projetado para atuar como destino central de push e fetch. Em vez de conter uma pasta de projeto editável, ele contém diretamente a estrutura interna do Git. Em laboratório, criar um bare repository em uma pasta local permite simular servidor sem depender de rede.

O git-shell é um shell restrito que permite operações Git sem conceder shell completo ao usuário no servidor. Em infraestrutura real, isso reduz superfície de ataque: uma conta dedicada para Git pode receber comandos necessários para push e pull, mas não executar comandos arbitrários no sistema. Mencionar git-shell ajuda você a perceber que colaboração envolve segurança de acesso e princípio do menor privilégio.

A segurança no uso de Git e GitHub não se limita a autenticação. Evite versionar segredos, tokens, chaves privadas, arquivos .env e credenciais. Compreenda que apagar um segredo de um commit posterior não remove necessariamente o segredo do histórico. Se uma chave foi publicada, deve ser revogada. A revisão de pull request deve incluir verificação de arquivos sensíveis.

Problemas comuns de infraestrutura incluem remote configurado com URL errada, falta de permissão, chave SSH não cadastrada, agente SSH não carregado, token expirado, branch sem upstream e divergência entre branch local e remota. As perguntas diagnósticas: qual é o remote? qual protocolo está sendo usado? há autenticação válida? a branch tem upstream? o usuário tem permissão no repositório? o repositório remoto existe?

```
# Repositório bare local simulando servidor mkdir servidor_remoto cd servidor_remoto
git init --bare projeto.git # Em outro diretório de trabalho git remote add origin /
caminho/para/servidor_remoto/projeto.git git push -u origin main # Teste de clonagem
em terceira pasta git clone /caminho/para/servidor_remoto/projeto.git copia-teste #
Geração de chave SSH moderna ssh-keygen -t ed25519 -C "seu-email@exemplo.com" #
Nunca publique a chave privada. # Cadastre apenas o conteúdo de id_ed25519.pub.
```

Diagnóstico de infraestrutura

Escolha o sintoma e veja as perguntas diagnósticas.

Selecione...

v

Aguardando seleção.

Conferi git remote -v.

Verifiquei protocolo usado: HTTPS ou SSH.

Confirmei permissão no GitHub.

Não publiquei chave privada ou token.

Auditoria de segurança antes do push

☐ Rodei `git status`.

☐ Conferi `git remote -v`.

☐ Verifiquei se não há `.env`, token, senha ou chave privada.

☐ Confirmei que a branch correta tem upstream.

10. Ecossistema de hospedagem: GitHub, GitLab, Bitbucket e GitWeb

10. Ecossistema de hospedagem: GitHub, GitLab, Bitbucket e GitWeb

Embora o módulo esteja inserido em uma trilha sobre GitHub, é importante compreender que GitHub é uma plataforma construída sobre Git, não o Git em si. O Git é o sistema de controle de versão; GitHub adiciona hospedagem, interface web, controle de acesso, issues, pull requests, wikis, actions, releases, páginas, discussões, estatísticas e recursos de colaboração. Essa distinção evita dependência conceitual da plataforma.

O GitHub destaca-se pela ampla adoção em projetos de código aberto, visibilidade de portfólio e ecossistema de colaboração. Para você, ele funciona como vitrine técnica, ambiente de aprendizagem e espaço de interação com comunidades. Pull requests, issues e README tornam o processo de desenvolvimento legível para avaliadores e colegas. No entanto, sua adoção deve vir acompanhada de responsabilidade sobre privacidade, licenciamento e qualidade do histórico público.

O GitLab possui forte orientação DevOps e oferece integração contínua, entrega contínua, registro de pacotes, gestão de issues e opção de auto-hospedagem. Organizações que precisam controlar infraestrutura e dados podem preferir GitLab self-managed. O conhecimento transferível é a base Git: clone, commit, branch, push, pull e merge continuam relevantes mesmo quando a interface de pull request recebe outro nome.

O Bitbucket é frequentemente associado ao ecossistema Atlassian, especialmente Jira e Confluence. Em disciplinas de graduação, pode ser menos comum que GitHub, mas serve para mostrar que diferentes organizações escolhem plataformas conforme integração com processos existentes.

O GitWeb é um visualizador web simples incluído no ecossistema Git, voltado à navegação de repositórios. Ele não oferece a riqueza de colaboração de plataformas modernas, mas é útil para compreender a separação entre versionamento e serviços adicionais.

A escolha de plataforma deve considerar critérios objetivos: visibilidade pública, privacidade, permissões, integração com ferramentas, suporte a CI/CD, custo, governança institucional, facilidade de uso e requisitos legais. Não confunda popularidade com adequação universal.

No projeto final do módulo, o GitHub será usado porque o tema da trilha é GitHub e porque a plataforma facilita visualização de branches, pull requests, histórico e README. Contudo, os conceitos aprendidos não ficam presos à plataforma — essa transferência de conhecimento amplia o valor formativo do curso.

Plataforma	Ênfase	Relevância para você
GitHub	Colaboração pública, pull requests, portfólio e ecossistema de código aberto.	Ideal para publicar projeto final, exercitar revisão e construir evidência pública.
GitLab	DevOps integrado, CI/CD e auto-hospedagem.	Ajuda a compreender ciclo de vida além do versionamento.
Bitbucket	Integração com Jira e ferramentas Atlassian.	Relevante para ambientes corporativos com rastreabilidade de demandas.
GitWeb	Visualização simples de repositórios Git.	Mostra que hospedagem pode ser separada de colaboração avançada.

Compare plataformas

GitHub

Forte em colaboração pública, pull requests, portfólio e ecossistema de código aberto.

GitLab

Forte em DevOps integrado, CI/CD e opção de auto-hospedagem.

Bitbucket

Comum em ambientes que usam Jira e Confluence.

GitWeb

Visualizador simples de repositórios Git.

Escolha de plataforma

- **Portfólio acadêmico público:** GitHub.
- **Controle institucional de infraestrutura:** GitLab.
- **Ambiente Atlassian:** Bitbucket.
- **Navegação web básica:** GitWeb.

Conclusão

Conclusão

A trilha valorizou a compreensão conceitual e evidência prática. Você não deve apenas saber executar git merge ou abrir um pull request; deve explicar quando usar merge, quando evitar rebase, como resolver conflito, por que proteger main e como documentar decisões. Essa capacidade reflexiva diferencia uso mecânico de competência profissional. Em graduação, tal competência é transferível para disciplinas de programação, ciência de dados, iniciação científica, extensão, produção textual colaborativa e projetos interdisciplinares.

O projeto final em Markdown sintetiza a proposta pedagógica. Ao criar um guia de aprendizado, você aplica Git e GitHub em uma tarefa significativa, registra histórico, revisa contribuições e produz material que pode beneficiar outros aprendizes. O repositório final é simultaneamente produto, portfólio e evidência de processo.

Por fim, o módulo reforçou que ferramentas de versionamento são também ferramentas de cultura profissional. Elas incorporam valores de transparência, rastreabilidade, revisão, colaboração, recuperação e melhoria contínua. Quando você aprende a usar branches com clareza, pull requests com respeito, rebase com responsabilidade e infraestrutura com segurança, desenvolve hábitos que ultrapassam o terminal. Desenvolve uma postura de engenharia: pensar antes de integrar, comunicar antes de alterar, registrar antes de esquecer e proteger o trabalho coletivo.

Checklist final de evidências

Repositório publicado no GitHub.

Branch criada com escopo claro.

Pull request aberto ou simulado.

Conflito criado, resolvido e documentado.

Decisão entre merge/rebase justificada.

Reflexão final escrita em Markdown.

Entrega final sugerida

Copie este roteiro para seu README ou relatório final.

```
## Evidências do Módulo 3
- Branch criada:
- Pull request:
- Estratégia de integração usada:
- Conflito resolvido:
- Decisão sobre merge ou rebase:
- Reflexão final:
```

Referências

Referências

CHACON, Scott; STRAUB, Ben. Pro Git. 2. ed. Tradução para português. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 18 ago. 2024.

CÓDIGO FONTE TV. Code Review no GitHub. YouTube, 2023. Disponível em: https://www.youtube.com/watch?v=2alg7MQ6_sl. Acesso em: 18 ago. 2024.

DUDLER, Roger. Git - Guia prático. 2023. Disponível em: https://rogerdudler.github.io/git-guide/index.pt_BR.html. Acesso em: 18 ago. 2024.

GITHUB. GitHub Docs. Disponível em: <https://docs.github.com/>. Acesso em: 18 ago. 2024.

Quiz final expandido - gabarito e feedback

1. O que é uma branch no Git?

- Uma cópia completa da pasta do projeto
- ✓ **Uma referência nomeada para um commit**
- Um backup automático no GitHub
- Um arquivo dentro de .gitignore

Feedback: Branch é uma referência leve que aponta para um commit.

2. Qual comando cria e alterna para uma branch em um único passo?

- git branch -d nome
- ✓ **git switch -c nome**
- git merge nome
- git log --all

Feedback: git switch -c nome cria a branch e troca para ela.

3. Quando ocorre detached HEAD?

- ✓ **Quando HEAD aponta diretamente para um commit**
- Quando a branch main é excluída
- Quando o GitHub fica offline
- Quando o README está vazio

Feedback: Detached HEAD ocorre quando HEAD não aponta simbolicamente para uma branch.

4. Quando o merge fast-forward é possível?

- Quando há conflito no mesmo arquivo
- ✓ **Quando a branch de destino não avançou desde a criação da origem**
- Quando usamos git reset --hard
- Quando o PR foi fechado

Feedback: No fast-forward, basta mover a referência da branch de destino.

5. O que deve ser feito após editar um arquivo em conflito?

- Deixar os marcadores no arquivo
- ✓ **Executar git add para marcar como resolvido**
- Executar git push --force
- Apagar a branch main

Feedback: Após resolver manualmente, git add informa ao Git que o conflito foi resolvido.

6. Qual é a regra ética central do rebase?

- Sempre aplicar rebase em main pública
- ✓ **Não fazer rebase em branch pública compartilhada sem acordo**
- Usar rebase para apagar autoria
- Trocar todos os merges por rebase

Feedback: Rebase reescreve hashes, por isso exige cuidado em branches compartilhadas.

7. Qual é a função de um pull request no GitHub?

- Apagar commits antigos
- ✓ **Transformar uma branch publicada em proposta de alteração revisável**
- Criar automaticamente chaves SSH
- Substituir o Git local

Feedback: Pull request é espaço de proposta, revisão, discussão e integração.

8. Qual fluxo é mais adequado para projeto simples com branches curtas e PRs?

- ✓ **GitHub Flow**
- Git Flow completo sempre
- Sem controle de versão
- GitWeb Flow

Feedback: GitHub Flow simplificado é adequado para ensino inicial, documentação e entregas frequentes.

Nota de corte: 75%.

Glossário expandido

Branch

Referência nomeada que aponta para um commit, representando uma linha de desenvolvimento.

HEAD

Referência especial que indica o contexto atual do repositório.

Detached HEAD

Estado em que HEAD aponta diretamente para um commit, não para uma branch.

Merge

Operação que integra alterações de uma branch em outra.

Fast-forward

Integração em que a branch de destino apenas avança para um commit descendente.

Conflito

Situação em que o Git não consegue combinar alterações automaticamente.

Rebase

Reaplicação de commits sobre uma nova base, reescrevendo hashes.

Pull request

Proposta de alteração em uma branch, com revisão e discussão no GitHub.

Remote

Apelido para outro repositório, como origin.

Bare repository

Repositório sem diretório de trabalho, usado como destino de push/fetch.